Taller: Sistema de agentes para el proyecto LatinosMC

Jhon Edinson Oliveros Tovar · Diplomado en Ingeniería de Software con IA y Sistemas Agénticos · FET · 09/11/2026

1. Proyecto y herramienta

Proyecto: LatinosMC es una plataforma de votaciones y ranking para servidores de Minecraft (`latinosmc.com`) cuyo producto real es la **credibilidad del ranking**: el jugador vota, el backend valida el voto con reglas antifraude, el plugin entrega la recompensa y el ranking se actualiza.

Herramienta del equipo: Claude Code (CLI en Windows con PowerShell). Para revisar cada pull request se usa además Claude en GitHub, con el workflow `claude-code-review.yml` que ya está en el repositorio. El sistema se describe en el vocabulario de Claude Code: `tools`, `disallowedTools`, `model`, `skills`, `hooks` y `Agent(...)`.

Repositorios:

- **Monorepo** github.com/JhonTova/Latinos-Minecraft-Web, con el backend en `backend/` (Java 21, Spring Boot, Maven multimódulo con 10 módulos, PostgreSQL + Flyway, JUnit 5, Mockito y ArchUnit) y el frontend en `frontend/` (Next.js 16, React 19, TypeScript, Tailwind 4).
- El plugin de Minecraft está en github.com/JhonTova/LatinosMC y queda fuera del sistema. Solo importa su contrato, las rutas `/api/v1/plugin/*`.
- Este diseño quedó versionado en el monorepo, en `docs/agentes/`, mediante el pull request #7.

Un dato que condiciona todo el diseño: `scripts/actualizar.sh` **despliega lo que hay en** `main`. Lo que llega a `main` llega a producción.

2. Sistema de agentes

2.1 Un sistema único para el monorepo, con agentes por carpeta

Elegí **un solo sistema**, con dos agentes compartidos (`analista` y `revisor`) y agentes propios de cada parte, cada uno limitado a `backend/**` o a `frontend/**`. Las razones:

- Backend y frontend viven en un repositorio, se integran con un solo pull request y una sola CI.
- El punto más frágil del proyecto es justamente el **contrato entre las dos partes**: las rutas del backend, la lista blanca `RUTAS_PERMITIDAS` del puente `/api/backend` y los tipos de `tipos.ts`. Con dos sistemas separados, nadie vería los dos lados del contrato en la misma revisión.
- Los stacks sí son distintos, y por eso las dos partes no tienen los mismos agentes. El backend tiene una suite de 329 pruebas y una prueba de arquitectura, así que hace TDD completo. El frontend no tiene pruebas automáticas y su riesgo está en lo visual, la accesibilidad y el puente, así que lleva un `verificador-ui` que mira la aplicación real.

Cómo se hacen cumplir los permisos en Claude Code. El campo `tools` decide *qué herramientas* tiene un agente, pero no *sobre qué carpetas* las usa. Por eso cada agente que edita declara en su frontmatter un **hook** `PreToolUse` con `matcher: "Edit|Write"`. Ese hook rechaza con código de salida 2 cualquier ruta fuera de las carpetas de su rol, y es una restricción real, no una instrucción. Otro hook con `matcher: "Bash"` deja ejecutar solo los comandos del rol (`mvn`, `npm`, `git`). En el `.claude/settings.json` del equipo, `permissions.deny` bloquea para todos `Read(.env)`, `Edit(docker-compose.yml)` y `Edit(scripts/**)`. Además:

- Ningún subagente tiene la herramienta `Agent`, así que no se invocan entre sí.
- Solo el orquestador conversa con el desarrollador. Los subagentes no tienen `AskUserQuestion`: devuelven sus preguntas en el informe.

Todos los agentes terminan con el mismo contrato de informe: `ESTADO · ARCHIVOS · EVIDENCIA · NECESITA DE OTRO AGENTE · PREGUNTAS PARA EL DESARROLLADOR`.

2.2 Tabla de agentes

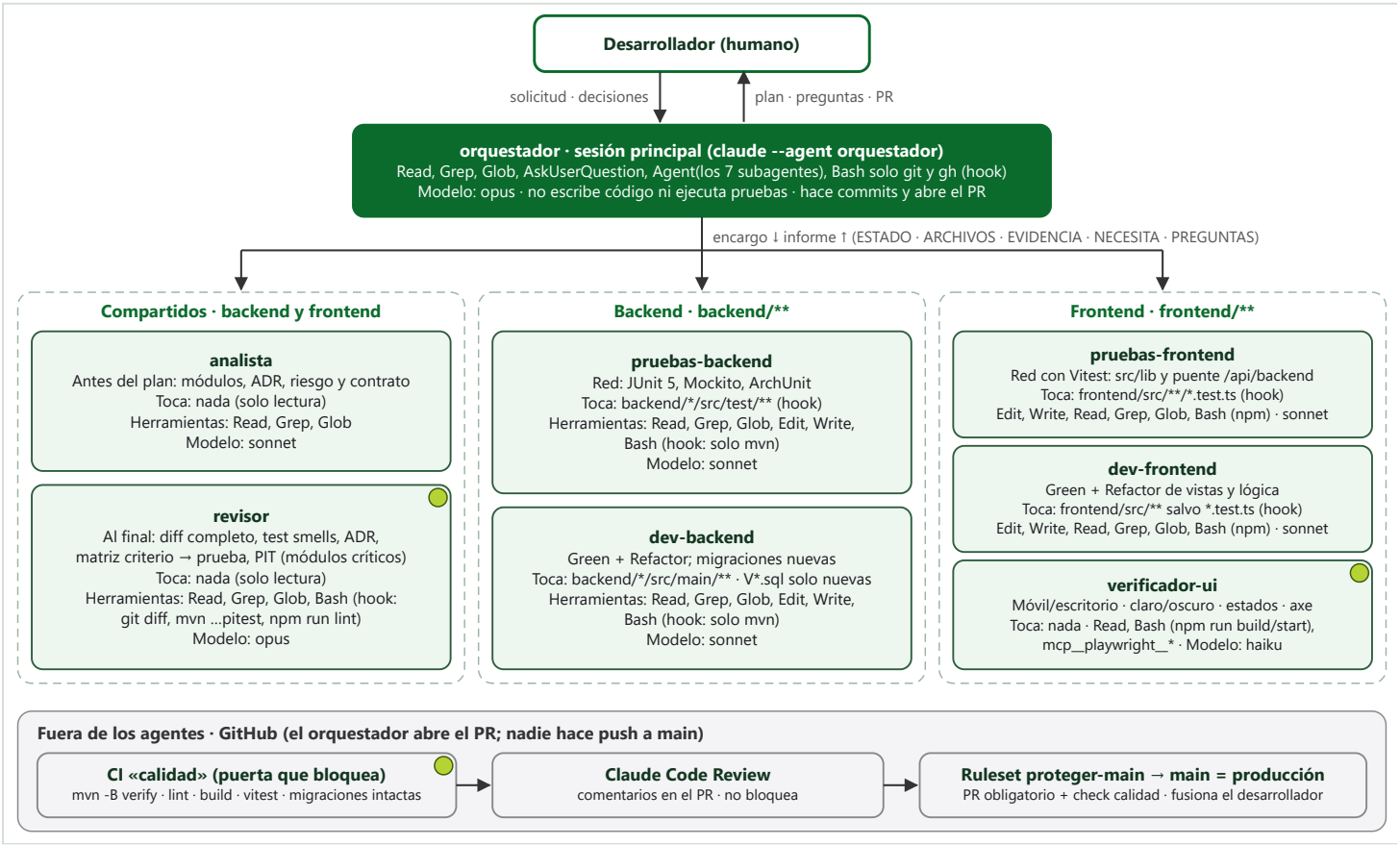
Nombre y rol	Cuándo actúa	Qué recibe y qué entrega	Qué puede tocar	Herramientas y permisos (Claude Code)	Modelo y por qué
orquestador *Orquestador · ambos.* Lleva la solicitud de principio a fin: decide qué subagente actúa, en qué orden, cuándo parar y cuándo preguntar. No escribe código.	Lo inicia el desarrollador con `claude --agent orquestador`. Está presente en todos los pasos.	**Recibe:** la solicitud con sus criterios de aceptación y las respuestas del desarrollador. **Entrega:** el plan, los encargos, un commit por bloque y el PR con la matriz criterio→prueba y la evidencia.	**Puede:** crear la rama, hacer commits, hacer push de la rama y abrir el PR. **No puede:** editar archivos, ejecutar `mvn` o `npm`, hacer push a `main` ni fusionar.	`tools: Read, Grep, Glob, AskUserQuestion, Bash, Agent(analista, pruebas-backend, dev-backend, pruebas-frontend, dev-frontend, verificador-ui, revisor)` · `disallowedTools: Edit, Write` · hook Bash: solo `git switch -c|add|commit|push origin <rama>` y `gh pr create|view`.	**opus.** Decide alcance, bucles y paradas, y es el único que habla con el humano: un error de juicio aquí se multiplica en todo el flujo. Se ejecuta una vez por solicitud, así que el costo está acotado.

analista *Subagente · ambos.* Hace el mapa del cambio antes del plan: módulos y su riesgo (🔴 `identity`, `voting`, `trust`, `billing`), los ADR que aplican y el contrato afectado.	Paso 2. Lo invoca el orquestador una vez por solicitud, y otra vez si el plan cambia de alcance.	**Recibe:** la solicitud y sus criterios. **Entrega:** módulos y archivos probables, ADR citados con su ruta, endpoints reales con `archivo:línea` y las decisiones de alcance que le tocan al humano.	Nada: es de solo lectura.	`tools: Read, Grep, Glob`	**sonnet.** Lee y cruza mucho texto (30 ADR y el código) con un juicio moderado. No investiga librerías nuevas, porque el stack está cerrado, así que no necesita opus ni web.
pruebas-backend *Subagente · backend.* Escribe las pruebas **antes** del código (Red) a partir de los criterios. En los módulos 🔴 cubre los casos de borde y los errores, y fija el tiempo con un `Clock` controlado.	Paso 4. Lo invoca el orquestador por cada bloque de criterios del backend, y otra vez si el revisor encuentra pruebas débiles.	**Recibe:** los criterios, el mapa del analista y las clases que hay que probar. **Entrega:** las pruebas, la salida de Maven con el fallo y su motivo (compilación o aserción) y la tabla criterio→prueba.	Solo `backend/*/src/test/**`. **No puede:** tocar `src/main`, los `pom.xml` ni las migraciones.	`tools: Read, Grep, Glob, Edit, Write, Bash` · hook Edit|Write: `backend/*/src/test/**` · hook Bash: `mvn -B -pl <módulo> -am test|verify`.	**sonnet.** Ejecuta criterios que ya están decididos y escribe buen código de prueba a un costo medio. Descarté haiku porque tiende a escribir pruebas que repiten lo que preparan.
dev-backend *Subagente · backend.* Escribe el código mínimo que hace pasar las pruebas (Green) y luego refactoriza. Crea migraciones Flyway nuevas con la skill `migracion-flyway`.	Paso 5, después de que el Red queda confirmado. Vuelve a actuar con los hallazgos de la CI o del revisor.	**Recibe:** las pruebas en rojo y el mapa. **Entrega:** el código, `mvn -B verify` en verde (incluye `ArquitecturaTest`) y la lista de archivos.	`backend/*/src/main/**` y migraciones **nuevas**. **No puede:** tocar `src/test`, editar un `V*.sql` existente, cambiar `application.yml` de producción ni agregar dependencias.	`tools: Read, Grep, Glob, Edit, Write, Bash` · `skills: migracion-flyway` · hook Edit|Write: `backend/*/src/main/**`, y rechaza un `V*.sql` que ya exista · hook Bash: solo `mvn`.	**sonnet.** La implementación está acotada por las pruebas y se repite en un bucle de hasta 3 vueltas, así que importan la velocidad y el costo.
pruebas-frontend *Subagente · frontend.* Hace el Red con Vitest para la lógica sin interfaz: `src/lib/**` y la lista blanca `RUTAS_PERMITIDAS` del puente `/api/backend`.	Paso 4. Solo actúa si la solicitud toca esa lógica o el puente; en un cambio puramente visual no se invoca.	**Recibe:** los criterios y las rutas nuevas que encontró el analista. **Entrega:** archivos `*.test.ts` en rojo con el motivo del fallo y la tabla criterio→prueba.	Solo `frontend/src/**/*.test.ts`. **No puede:** tocar el resto de `src` ni `package.json`.	`tools: Read, Grep, Glob, Edit, Write, Bash` · hook Edit|Write: `frontend/src/**/*.test.ts` · hook Bash: `npm run test`.	**sonnet.** Por las mismas razones que `pruebas-backend`.
dev-frontend *Subagente · frontend.* Implementa vistas y lógica en Next.js 16 siguiendo la guía local `node_modules/next/dist/docs`. Usa solo los tokens de `globals.css` y cubre los estados cargando, vacío, error y sin permisos.	Paso 5, tras el Red, o directo si el cambio es solo visual. Vuelve a actuar con los hallazgos de `verificador-ui` o del revisor.	**Recibe:** los criterios, el mapa y las pruebas. **Entrega:** el código y la salida de `npm run lint`, `build` y `test`.	`frontend/src/**` salvo `*.test.ts`, y `frontend/public/**`. **No puede:** tocar `package.json` ni el lock, ni llamar al backend fuera del puente.	`tools: Read, Grep, Glob, Edit, Write, Bash` · `skills: vercel-react-best-practices` · hook Edit|Write por carpeta · hook Bash: `npm ci|run lint|build|test`.	**sonnet.** Next 16 cambió APIs respecto a lo que el modelo conoce: necesita leer documentación y aplicarla, no solo completar patrones.
verificador-ui *Subagente · frontend.* Comprueba la pantalla real, no el código: cada vista afectada a 390 px y en escritorio, en tema claro y oscuro. Revisa los estados, axe-core, la consola y que `/panel/admin` siga respondiendo 404 a quien no es administrador.	Paso 5, cuando `dev-frontend` deja lint y build en verde. Se repite en cada vuelta.	**Recibe:** las vistas y rutas que debe revisar. **Entrega:** 4 capturas por vista, las violaciones de axe con su regla WCAG, los errores de consola y un veredicto.	Nada: no edita.	`tools: Read, Bash, mcp__playwright__*` · `disallowedTools: Edit, Write` · hook Bash: `npm run build|start`.	**haiku.** Sigue una lista fija muchas veces por solicitud (vistas × 4). El veredicto de accesibilidad lo da axe de forma determinista, así que el modelo solo reporta: conviene que sea barato y rápido.

revisor *Subagente · ambos.* Es un segundo par de ojos, independiente de quien escribió. Revisa los test smells, que la matriz criterio→prueba esté completa, los ADR (fronteras entre módulos, ADR-0019, ledger inmutable, `Clock`) y el contrato. Además ejecuta **mutación con PIT** en los módulos 🔴 tocados.	Paso 6, sobre el diff completo de la rama antes del PR, y otra vez tras cada vuelta de correcciones.	**Recibe:** `git diff main...HEAD`, los criterios y el mapa. **Entrega:** hallazgos con severidad, `archivo:línea`, la regla y el rol que debe corregir, más los mutantes supervivientes clasificados.	Nada: es de solo lectura.	`tools: Read, Grep, Glob, Bash` · `disallowedTools: Edit, Write` · hook Bash: `git diff|log`, `mvn …pitest:mutationCoverage` y `npm run lint`.	**opus.** Su trabajo es juicio puro: si una prueba fallaría con el código mal escrito, o si un mutante es equivalente. Corre entre 1 y 3 veces por solicitud, un costo pequeño frente a un error en producción.

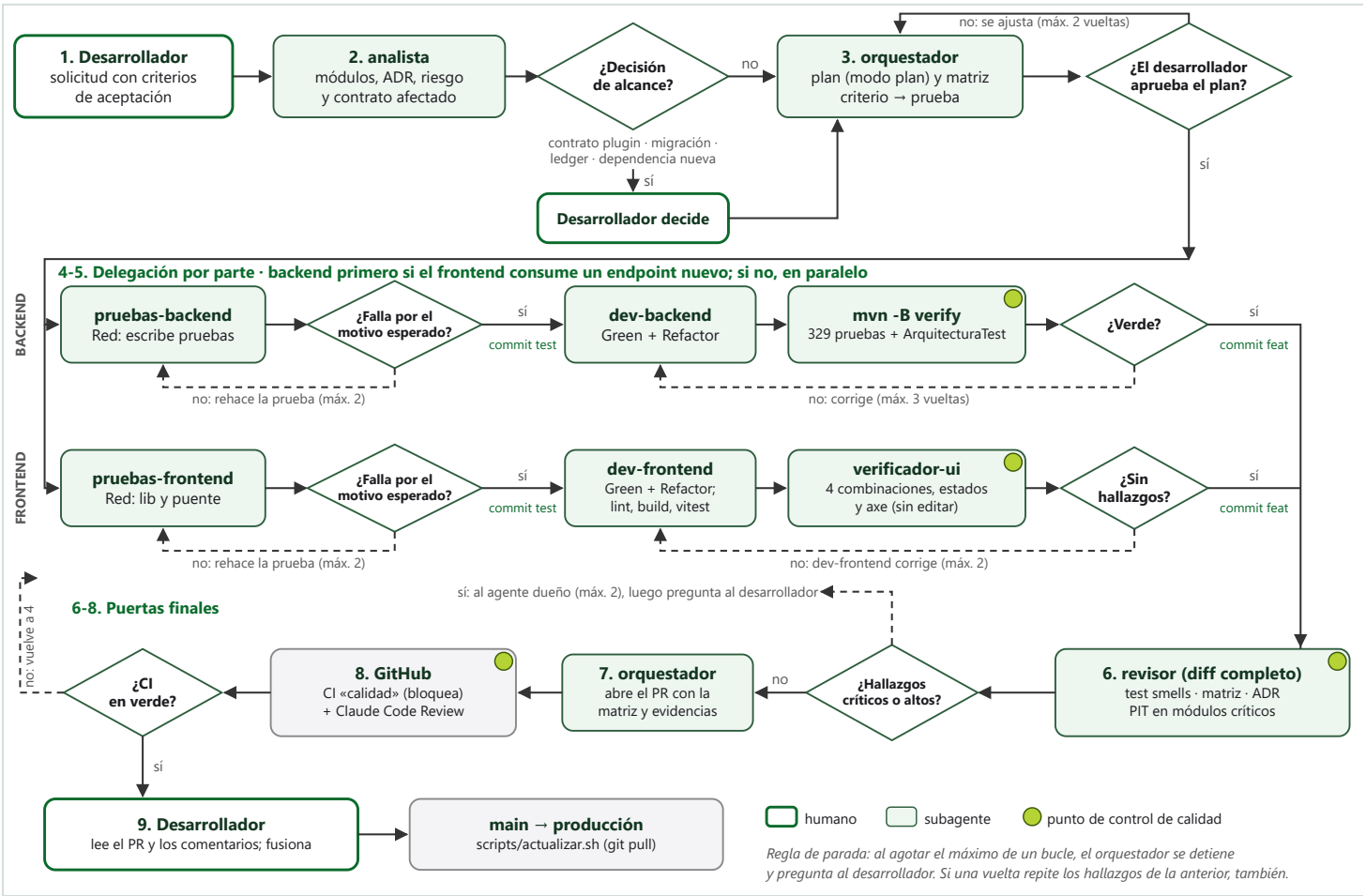
Los hooks (`limitar-rutas`, `limitar-bash` y la regla de migraciones) son parte del diseño. Siguiendo el taller, aquí no se escriben los archivos de los agentes ni de los hooks.

2.3 Diagrama de estructura



Estructura: el desarrollador habla solo con el orquestador, que delega en 7 subagentes agrupados por carpeta. Los subagentes no se comunican entre sí. Debajo están las puertas de GitHub, que quedan fuera de los agentes.

2.4 Diagrama de flujo



Flujo de una solicitud: dos decisiones humanas antes de escribir código, carriles de backend y de frontend con bucles limitados, revisión independiente, CI y fusión humana.

Cómo leer el flujo:

- El desarrollador decide en tres momentos: la decisión de alcance (contrato del plugin, migración, ledger o dependencia nueva), la aprobación del plan y la fusión del PR.
- Cada bucle tiene un máximo de vueltas. Al agotarlo, o si una vuelta repite los hallazgos de la anterior, el orquestador se detiene y pregunta.
- Si el frontend consume un endpoint nuevo, el carril del backend va primero; si no, los dos carriles corren en paralelo.

3. Calidad del código escrito por la IA

La idea central: **quien escribe nunca se aprueba a sí mismo**. Las pruebas las escribe un agente que no puede tocar el código; el código, otro que no puede tocar las pruebas; y lo verifican un agente que no edita y una CI en una máquina limpia. Cada problema típico tiene al menos una puerta. La tabla dice cuál es, cuándo actúa, qué evidencia deja y si bloquea.

Problema típico	Qué lo evita o lo detecta	Cuándo	Evidencia	¿Bloquea?
Prueba vacía, tautológica o que solo verifica el mock (B y F)	Roles separados por hook: quien prueba no implementa. Antes del Green, `pruebas-*` confirma el Red con el motivo del fallo. El `revisor` aplica la lista de test smells y, en los módulos 🔴, PIT: una prueba que no falla al invertir una condición deja un mutante vivo.	Pasos 4 y 6	Salida de `mvn` o `vitest` en rojo antes del código; el commit `test(...)` va antes que `feat(...)`; informe de PIT con los supervivientes clasificados.	Sí, en el flujo: sin el Red no se delega el Green, y un superviviente sin clasificar en un módulo 🔴 o un hallazgo alto impiden abrir el PR.
Criterio de aceptación sin prueba	El orquestador arma en el plan la matriz criterio→prueba, cada `pruebas-*` la completa y el `revisor` marca como hallazgo alto cada criterio que quede sin prueba.	Pasos 3, 4 y 6	La matriz en la descripción del PR.	Sí (flujo).
Código sin prueba (B)	JaCoCo en la CI: 80 % de líneas como **suelo** en los módulos 🔴 tocados; en el resto es solo un informe. La cobertura no prueba calidad: para eso está PIT.	CI	Informe de JaCoCo como artefacto.	Sí en los módulos 🔴; en el resto, no.

API o método Java/Spring inventado	El compilador en `mvn -B verify`, que ejecuta `dev-backend` y repite la CI.	Paso 5 y CI	Log de Maven.	**Sí** (CI).
Endpoint o campo inventado (F)	El `analista` entrega los endpoints reales con `archivo:línea`. Una ruta fuera de `RUTAS_PERMITIDAS` responde 404. `pruebas-frontend` prueba el puente, y `npm run build` revisa los tipos de `tipos.ts`.	Pasos 2, 4 y 5, y CI	Informe del analista y salida de `vitest` y `build`.	**Parcial:** un campo inventado dentro de `tipos.ts`, que se escribe a mano, no lo ve nadie (es un límite).
API de Next.js que cambió o no existe	`dev-frontend` lee la guía local de Next 16, y `npm run lint` y `build` fallan si algo no existe.	Paso 5 y CI	Log del build.	**Sí** (CI).
Romper la arquitectura (un módulo importa a otro)	`ArquitecturaTest` (ArchUnit) corre dentro de `mvn verify`. Las lecturas SQL entre módulos fuera de ADR-0019 y los usos de `Instant.now()` los revisa el `revisor`; se propone además una regla de ArchUnit para `Instant.now()`.	Pasos 5 y 6, y CI	Resultado de `ArquitecturaTest` y hallazgo con `archivo:línea`.	**Sí** (CI) para las importaciones; el SQL lo bloquea el revisor.
Editar una migración ya aplicada o escribir en el ledger	Un hook de `dev-backend` rechaza un `V*.sql` que ya existe. En la CI, el job «migraciones intactas» falla si `git diff --diff-filter=M origin/main` toca `db/migration/`. El `revisor` busca `UPDATE` o `DELETE` sobre el ledger.	Paso 5 y CI	El rechazo del hook y el job en rojo.	**Sí.**
Interfaz rota, inaccesible o sin tema oscuro (F)	`verificador-ui` revisa 4 combinaciones por vista, axe-core, la consola y los estados. ESLint (con `eslint-config-next`) revisa la accesibilidad en el JSX.	Paso 5 y CI	Capturas, reporte de axe con su regla WCAG y lint.	**Sí** (flujo): una violación *serious* o *critical* de axe devuelve el trabajo a `dev-frontend`.
Fuga de seguridad: proxy abierto o revelar que `/panel/admin` existe	`pruebas-frontend` fija con pruebas que una ruta no listada recibe 404. `verificador-ui` comprueba el 404 de admin, y el `revisor` revisa cada patrón nuevo de la lista blanca.	Pasos 4 a 6, y CI	`vitest` y capturas.	**Sí.**
Romper el contrato del plugin `/api/v1/plugin/*`	El `analista` lo detecta, el orquestador para y pregunta, y el PR lo declara.	Paso 2 y PR	La decisión queda en el PR.	La decide el desarrollador.
Código que nadie ejecutó	La CI `calidad` corre en una máquina limpia en cada PR, y el ruleset `proteger-main` exige PR y CI en verde. Claude Code Review comenta el PR, y el desarrollador clasifica cada comentario como aceptar, cuestionar o descartar.	Pasos 8 y 9	La ejecución de Actions y los comentarios en el PR.	**Sí** la CI con el ruleset; **no** la revisión de IA.

Qué existe hoy y qué incorpora el sistema.

- **Ya funciona:** la CI (`mvn -B verify` con 329 pruebas y `ArquitecturaTest`, y en el frontend `lint` y `build`), Claude Code Review en cada PR y las dos skills.
- **Ruleset `proteger-main`, activo desde el 09/11/2026:** exige PR (no se puede hacer push directo a `main`) y que pasen los checks `cambios`, `backend` y `frontend`. Además impide borrar `main` o hacerle force push, y no tiene excepciones, ni siquiera para el dueño del repositorio. Lo activé al diseñar el sistema porque la API de GitHub mostraba `main` sin protección: hasta entonces la CI solo informaba. Claude Code Review no es un check obligatorio porque es una opinión que clasifica el desarrollador.
- **Lo agrega este diseño:** los hooks de permisos, Vitest, PIT y JaCoCo en el backend, y el job «migraciones intactas», que se sumará a los checks obligatorios del ruleset. Vitest es una dependencia nueva, así que pasa por la decisión humana y se registraría en un ADR.

Lo que el sistema no alcanza a detectar:

- **SQL que nunca se ejecutó.** Ninguna prueba usa PostgreSQL real (es la deuda de ADR-0021), así que una consulta mal escrita o una migración que falla en Postgres pasa por todas las puertas y aparece recién en producción.
- **Concurrencia en el motor de votos.** Dos votos simultáneos y la idempotencia bajo carga no los ejercita ninguna prueba unitaria.
- **Un criterio de aceptación equivocado.** El sistema prueba lo que se pidió, no lo que debió pedirse.
- **Que el Red falle por la razón correcta.** El orden de los commits demuestra que la prueba existía antes que el código, pero no por qué fallaba: eso lo declara el agente y lo juzga el revisor.
- **Lo que requiere juicio humano.** Los mutantes equivalentes y la calidad visual fina (jerarquía, textos, estética) que axe no mide; haiku puede pasarlos por alto.
- **Componentes React sin pruebas.** Solo la lógica y el puente tienen pruebas; la interfaz depende del verificador.
- **Tipos que se desalinean.** `tipos.ts` se escribe a mano y puede separarse del backend, porque no se genera desde OpenAPI.
- **Lo que se haga sin los agentes.** Los hooks solo actúan en las sesiones de Claude Code. Quien trabaje a mano solo encuentra la CI y el ruleset.

4. Diferencias con el sistema de referencia

Qué revisé de la referencia:

- `.github/agents/` de `777wills/mi-restaurante-ms` en la rama `calidad-agentes`, con 5 agentes (architect, test-developer, developer, reviewer y spec-auditor), y el preset `implement-orchestrator`.
- `speckit-doc.md`: la calidad está en la sección 15 y los agentes en la **17**.
- En `mi-restaurante-ui` la rama `calidad-agentes` no existe: los mismos agentes están en la rama `001-project-baseline`.

Lo que conservo y no repito aquí: el patrón orquestador → subagentes sin comunicación entre ellos, el contrato de informe, el TDD con roles separados, un revisor de solo lectura con test smells, y opus para el juicio con sonnet para la ejecución.

#	Referencia	Mi sistema	Por qué (razón del proyecto)
1	**Dos sistemas idénticos**, uno por repositorio. El orquestador es un preset de Spec Kit que se antepone a `/speckit-implement`.	**Un sistema** para el monorepo, con agentes por carpeta. El orquestador es el agente de la sesión principal (`claude --agent`) con la lista `Agent(...)` de los subagentes que puede llamar.	Backend y frontend comparten repositorio, PR y CI, y el contrato entre ellos se rompe justo en el PR que toca los dos. El proyecto no usa Spec Kit: sus requisitos viven en `docs/requisitos` y en 30 ADR.
2	«Qué puedes tocar» es **solo una instrucción**: `test-developer` y `developer` tienen `edit` sobre todo el repositorio.	**Hooks** `PreToolUse` en el frontmatter rechazan `Edit` y `Write` fuera de la carpeta del rol, y otros hooks limitan `Bash` a los comandos de cada rol.	`main` se despliega directo a producción. Una instrucción no es un permiso: con los hooks, el TDD estructural deja de depender de que el modelo obedezca.
3	`architect` usa opus con web y context7, solo en la Fase 0 del plan, para investigar versiones y alternativas.	`analista` usa sonnet, sin web, en **cada** solicitud, y mapea módulos, riesgo, ADR y contrato.	El stack está cerrado y `AGENTS.md` prohíbe agregar dependencias sin pedirlo. Las decisiones ya están en los ADR: el riesgo real es contradecir uno, no elegir mal una librería.
4	`spec-auditor` compara el código con `spec.md` y `tasks.md`.	**No existe.** La matriz criterio→prueba va en el PR y la valida el revisor.	No hay `spec.md` ni `tasks.md`, y las solicitudes son pequeñas. Un agente más solo duplicaría el trabajo del revisor.
5	Frontend con TDD completo en Vitest y cobertura del 80 %. El propio `developer` hace la verificación visual.	Pruebas solo para la lógica y el puente. La verificación visual la hace `verificador-ui`, con haiku y sin poder editar.	El frontend hoy no tiene pruebas y sus paneles cambian seguido. El riesgo de seguridad está en el puente y el de calidad en lo visual y lo accesible. Separar la verificación de quien implementa evita que se apruebe a sí mismo.
6	Mutación con Stryker **informativa** sobre todos los archivos de la feature; nunca bloquea.	PIT **solo en los módulos** 🔴 (`identity`, `voting`, `trust`, `billing`). Un superviviente sin clasificar impide abrir el PR.	El producto es la credibilidad del ranking: un mutante vivo en el antifraude es un fraude que pasaría. Correr PIT sobre los 10 módulos sería demasiado lento.
7	La referencia reconoce como límite que el Red «lo declara el propio agente» y que no se hace un commit por tarea.	El orquestador hace un commit `test(...)` antes de cada `feat(...)`, por bloque.	Así la evidencia del TDD es objetiva y se puede revisar en el PR. El equipo ya usa Conventional Commits y PR pequeños.
8	Umbral **global** de cobertura del 80 % que bloquea.	JaCoCo con un suelo del 80 % solo en los módulos 🔴 tocados; en el resto, solo un informe.	Las 329 pruebas actuales nunca midieron cobertura. Un umbral global pondría la CI en rojo desde el primer día, como pasó con Spotless, y empujaría a escribir pruebas para subir la cifra.
9	El desarrollador responde las preguntas de los agentes y clasifica la revisión de Copilot.	Además hay **paradas obligatorias**: contrato del plugin, migración, ledger y dependencia nueva. La fusión siempre la hace el humano, y la revisión en GitHub es de Claude Code Review.	El plugin vive en otro repositorio y un cambio de contrato exige publicar una release. Una migración aplicada no se deshace en producción. La herramienta del equipo es Claude Code y su workflow ya está en el repositorio.